Data Preparation for Machine Learning

From Data Collection to Results Presentation

Data preparation for Machine Learning is the process of transforming raw data into a clean, consistent, structured, and model-ready dataset. Unlike Business Intelligence, where the main goal is often to understand and report business performance, Machine Learning preparation focuses on producing data that allows a model to learn patterns and make reliable predictions.

Overall Workflow

```
Problem Definition → Data Collection → Data Understanding → Data Cleaning →
Data Integration → Data Transformation → Exploratory Data Analysis →
Feature Engineering → Feature Selection → Data Splitting → Preprocessing →
Class Balancing → Model Training → Evaluation → Interpretation →
Results Presentation
```

Table of Contents

1. Define the Machine Learning Problem

Before collecting data, clearly define what the model is expected to do.

Questions to Answer

- What problem are we solving?
- What is the prediction target?
- Is this classification, regression, clustering, or another task?
- What observations represent one sample?
- What information will be available when making a prediction?
- What metric will determine success?

Examples

Task Type	Example
Classification	Predict whether a customer will default: Yes / No
Multiclass classification	Identify a bird species: Quelea, African Golden Weaver, Black-Headed Weaver, Other
Regression	Predict house price
Time-series forecasting	Predict tomorrow's electricity demand

2. Define the Target Variable

The target variable is what the Machine Learning model is expected to predict.

Features	Target
Age	Disease
Income	Disease
Blood Pressure	Disease
Cholesterol	Disease

```
X = df.drop("disease", axis=1)
y = df["disease"]
```

- X = input features
- y = target / label

> ***Note:*** *This distinction is fundamental — the target should never accidentally be used as an input feature.*

3. Data Collection

Collect data relevant to the ML problem.

Possible Sources

- Kaggle
- Company databases
- SQL databases
- APIs
- Sensors
- IoT devices
- Surveys
- Images
- Audio
- Video
- Satellite data
- Government datasets
- Web data

Examples

```
Camera → Bird Images → Image Dataset
```

```
Microphone → Audio Recordings → Audio Dataset
```

```
Database → CSV/SQL → Machine Learning Dataset
```

4. Data Understanding

After collecting the data, inspect its structure.

```
df.head()
df.tail()
df.shape
df.columns
df.info()
df.dtypes
df.describe()
```

Questions to Answer

- How many samples are available?
- How many features?

- What are the data types?
- What does each feature represent?
- What is the target variable?
- Are there missing values?
- Are there duplicates?
- Are there unusual values?
- Is the target balanced?

5. Data Quality Assessment

Before cleaning, systematically evaluate data quality.

Quality Issue	Example
Missing data	Age = NaN
Duplicate data	Same record appears twice
Invalid values	Age = -20
Wrong data type	Date stored as text
Inconsistent values	Male, male, M
Outliers	Salary = 10,000,000
Incorrect labels	Bird image assigned wrong species

> ***Note:*** *The objective is to identify problems before they affect model training.*

6. Data Cleaning

Data cleaning corrects or removes problematic observations.

6.1 Missing Values

Check:

```
df.isnull().sum()
```

Possible strategies:

Remove records — df.dropna()

```
df.dropna()
```

Fill numerical values — df["age"].fillna(df["age"].median())

```
df["age"].fillna(df["age"].median())
```

Fill categorical values — df["gender"].fillna(df["gender"].mode()[0])

```
df["gender"].fillna(df["gender"].mode()[0])
```

> ***Note:*** *The choice depends on the dataset and the ML problem.*

7. Remove Duplicate Records

Duplicate observations can cause some examples to receive excessive influence during training.

```
df.duplicated().sum()
```

Remove duplicates:

```
df = df.drop_duplicates()
```

> ***Note:*** *Duplicates should be investigated first — some repeated observations may be legitimate.*

8. Correct Data Types

Machine Learning algorithms require appropriate representations. For example:

```
df["date"] = pd.to_datetime(df["date"])
```

Categorical variables may need to be encoded. Numerical variables should be stored as numerical types.

```
df["age"] = pd.to_numeric(df["age"], errors="coerce")
```

9. Handle Inconsistent Data

For categorical variables, standardize values. For example:

Before	After
Male / male / MALE / M	Male

```
df["gender"] = (
    df["gender"]
    .astype(str)
    .str.strip()
    .str.lower()
)
```

Then map values to a standard representation.

10. Detect and Handle Outliers

Outliers are unusually large or small observations. Examples:

- Age = 250
- Salary = 10 billion
- Temperature = -500°C

Detection Methods

Category	Methods
Statistical	IQR, Z-score
Visualization	Box plots, Histograms, Scatter plots

```
Q1 = df["income"].quantile(0.25)
Q3 = df["income"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
    (df["income"] < lower) |
    (df["income"] > upper)
]
```

> ***Note:*** *Do not automatically delete every outlier — an outlier may be a genuine and important observation.*

11. Data Integration

Sometimes data comes from multiple sources. For example:

```
Customer Dataset + Transaction Dataset + Product Dataset → Integrated ML
Dataset
```

Pandas methods include:

- pd.merge()
- pd.concat()
- df.join()

```
df = customers.merge(
    transactions,
    on="customer_id",
```

```
    how="inner"
)
```

> ***Note:*** *The integration process must ensure that records are correctly matched.*

12. Exploratory Data Analysis (EDA)

EDA helps understand patterns before modeling.

Type	Description	Example
Univariate	Analyze one variable	Distribution of age
Bivariate	Analyze two variables	Age versus disease
Multivariate	Analyze multiple variables	Age, income, education and occupation versus customer churn

Typical Visualizations

- Histograms
- Box plots
- Bar charts
- Scatter plots
- Correlation heatmaps

13. Analyze the Target Variable

The target requires special attention.

For Classification

```
df["target"].value_counts()
```

Calculate proportions:

```
df["target"].value_counts(normalize=True)
```

Class	Proportion
Class 0	90%
Class 1	10%

> ***Note:*** *This indicates class imbalance.*

For Regression, Inspect

- Distribution

- Minimum
- Maximum
- Mean
- Median
- Outliers

14. Feature Engineering

Feature engineering creates useful input variables from existing data. This is often one of the most important parts of ML preparation.

Example: Date

From: 2026-08-14, create:

- Year = 2026
- Month = 8
- Day = 14
- Day_of_week = Friday

Example: Sales

From Quantity and Unit Price, create:

```
Revenue = Quantity × Unit Price
```

Example: Text

Convert text into numerical representations using methods such as:

- TF-IDF
- Bag of Words
- Embeddings

Example: Images

- Resizing
- Normalization
- Cropping
- Augmentation

Example: Audio

- Resampling
- Noise reduction

- Normalization
- Feature extraction
- Spectrogram/MFCC generation

15. Feature Selection

Not every available variable should necessarily be used. Remove features that are:

- Irrelevant
- Redundant
- Constant
- Highly noisy
- Data leakage sources
- Identifiers without predictive meaning

For example, from the set: customer_name, customer_id, address, age, income, purchase_history —

> ***Note:*** *customer_id may identify a customer but may not provide meaningful predictive information.*

16. Prevent Data Leakage

Data leakage occurs when information that would not be available at prediction time accidentally enters the model's training data.

Example: Suppose we want to predict whether a customer will default. Candidate features are Customer Income, Credit Score, Loan Amount, and Default Status.

> ***Note:*** *If Default Status is the target, it must not be included as an input feature.*

Another common leakage problem occurs when preprocessing is performed on the entire dataset before splitting.

Correct principle: Split first, then fit preprocessing only on the training data.

17. Separate Features and Target

For supervised learning:

```
X = df.drop("target", axis=1)
y = df["target"]
```

Variable	Contains

X	Input variables
y	Prediction target

Example

X	y
Age, Income, Education, Experience	Salary

18. Train-Test Split

The dataset should be divided into subsets.

```
Complete Dataset
    ├── Training Set
    ├── Validation Set
    └── Test Set
```

A common approach is:

Split A	Split B
70% Training / 15% Validation / 15% Testing	80% Training / 20% Testing

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

> ***Note:*** *For time-series data, do not randomly shuffle the observations if that would allow future information to influence the past.*

19. Handle Class Imbalance

For classification problems, classes may not have equal numbers of observations.
Example:

Class	Count
Normal	9,000
Fraud	1,000

Possible Approaches

Approach	Description
Undersampling	Reduce the majority class
Oversampling	Increase the minority class
SMOTE	Generate synthetic minority examples
Class weights	Give greater importance to minority classes

```
class_weight="balanced"
```

> ***Note:*** *The appropriate strategy depends on the problem.*

20. Encode Categorical Variables

Most ML algorithms require numerical inputs. Suppose a Gender column contains: Male, Female, Male, Female.

One-hot encoding can produce:

- Gender_Male
- Gender_Female

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
```

For ordinal variables such as Low, Medium, High — an ordinal encoding may be appropriate if the order is meaningful.

21. Feature Scaling

Some algorithms are sensitive to feature magnitude. Example:

Feature	Range
Age	20–80
Income	10,000–500,000

> ***Note:*** *Income has a much larger numerical scale.*

Standardization

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

Transforms approximately to: mean = 0, standard deviation = 1

Min-Max Scaling

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
```

Typically transforms values to the 0–1 range.

Scaling is particularly important for models such as Logistic Regression, KNN, SVM, and Neural Networks.

> ***Note:*** *Tree-based models generally do not require feature scaling.*

22. Build a Preprocessing Pipeline

A robust ML workflow should avoid manually applying transformations inconsistently. Scikit-learn pipelines can combine preprocessing and modeling.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_features = ["age", "income"]
categorical_features = ["gender", "region"]

preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            StandardScaler(),
            numeric_features
        ),
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features
        )
    ]
)
```

Then combine preprocessing with a model:

```
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("preprocessing", preprocessor),
    ("classifier", LogisticRegression())
])
```

> ***Note:*** *This helps reduce leakage and keeps preprocessing reproducible.*

23. Data Augmentation for Images/Audio

For multimedia ML projects, data preparation may include augmentation.

Image Augmentation

- Rotation
- Flipping
- Cropping
- Zoom
- Brightness adjustment
- Translation

Audio Augmentation

- Background noise
- Time shifting
- Pitch modification
- Time stretching
- Volume changes

> ***Note:*** *Augmentation should produce realistic variations rather than unrealistic samples.*

24. Prepare the Final ML Dataset

After cleaning and transformation, the dataset should be checked again. Verify:

- No unexpected missing values
- Correct data types
- Correct labels
- No accidental target leakage
- Appropriate class distribution
- Features in correct format
- Training and test data separated correctly

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

25. Model Training

Once the data is ready, the prepared training data can be used to train models.

```
Training Data → Preprocessing → Machine Learning Algorithm → Trained Model
```

Task	Possible Models
Classification	Logistic Regression, Decision Tree, Random Forest, SVM, KNN, XGBoost, Neural Networks, CNN
Regression	Linear Regression, Decision Tree Regression, Random Forest Regression, Gradient Boosting, Neural Networks

26. Model Validation

The validation set can be used to compare models and tune hyperparameters. Example:

Model	Score
Model A	82%
Model B	87%
Model C	91%

> ***Note:*** *The analyst may investigate why Model C performs better.*

Techniques

- Cross-validation
- Grid Search
- Random Search
- Bayesian optimization

27. Model Evaluation

Evaluation depends on the ML task.

Classification — Key Metrics

Metric	Meaning
Accuracy	Percentage of correct predictions
Precision	Of the observations predicted positive, how many were actually positive?
Recall	Of the actual positive observations, how many were correctly identified?
F1-score	Balances precision and recall
ROC-AUC	Measures ranking/discrimination performance across thresholds

Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

28. Regression Evaluation

For regression problems, common metrics include:

- MAE
- MSE
- RMSE
- R^2

Example

Metric	Value
MAE	5.2
RMSE	7.1
R^2	0.84

> ***Note:*** *Interpretation depends on the scale and context of the target.*

29. Model Comparison

If multiple models are trained, compare them using the same evaluation procedure.

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.84	0.82	0.80	0.81
Random Forest	0.89	0.88	0.87	0.87
SVM	0.87	0.86	0.85	0.85

> ***Note:*** *Do not choose a model based on accuracy alone when the classes are imbalanced or when different errors have different costs.*

30. Model Interpretation

A Machine Learning result should be interpretable where possible.

Questions

- Which features are important?
- Which variables influence predictions?
- Which observations are difficult to classify?
- Where does the model make mistakes?

Possible Techniques

- Feature importance
- Permutation importance
- SHAP
- Partial dependence
- Confusion matrix
- Error analysis

31. Error Analysis

Do not only examine the overall metric — investigate incorrect predictions. Example:

Actual Bird	Predicted Bird
Quelea	Weaver
Weaver	Quelea
Quelea	Other

Ask

- Why were these examples misclassified?
- Are labels correct?
- Are classes visually similar?
- Is there insufficient training data?
- Are some classes underrepresented?
- Is the input quality poor?

> ***Note:*** *Error analysis can lead to improvements in the dataset itself.*

32. Results Visualization

Results should be presented using clear visualizations.

Task	Recommended Visuals
Classification	Confusion matrix, ROC curve, Precision-recall curve, Class distribution, Training/validation curves
Regression	Actual vs predicted plot, Residual plot, Error distribution
Deep Learning	Training loss, Validation loss, Training accuracy, Validation accuracy

33. Results Interpretation

The analyst should explain what the results mean. For example:

> ***Note:*** *The Random Forest model achieved an F1-score of 0.87, outperforming Logistic Regression at 0.81. The model therefore provides stronger overall classification performance on the test dataset.*

But do not stop there — also discuss:

- Strengths
- Weaknesses
- Errors
- Dataset limitations
- Generalization
- Practical implications

34. Results Presentation

A good ML presentation can follow this structure.

Slide	Content
1 — Project Title	e.g., Machine Learning Model for Bird Species Classification
2 — Problem Statement	What problem are you solving?
3 — Objectives	What does the project aim to achieve?
4 — Data Collection	Data source, number of samples, classes/features, collection method

Slide	Content
5 — Dataset Description	Number of records, features, target, class distribution
6 — Data Preparation	Missing values, duplicates, cleaning, transformation, encoding, scaling, augmentation
7 — EDA	Important patterns and distributions
8 — Dataset Splitting	Dataset → Training / Validation / Testing
9 — Model Development	Prepared Data → Preprocessing → Model → Training → Validation
10 — Model Performance	Accuracy, Precision, Recall, F1, ROC-AUC
11 — Error Analysis	Confusion matrix, misclassified examples, major error patterns
12 — Model Comparison	Compare different models
13 — Key Findings	Summarize important results
14 — Recommendations	What should be improved or implemented
15 — Conclusion	Summarize the complete project

35. Complete Machine Learning Data Preparation Workflow

You can use this as the main framework for teaching:

```
MACHINE LEARNING DATA ANALYSIS WORKFLOW
│
├── 1. Problem Definition
│       ├── Problem statement
│       ├── Objectives
│       ├── ML task
│       └── Evaluation metric
│
├── 2. Data Collection
│       ├── Kaggle
│       ├── Database
│       ├── API
│       ├── Sensors
│       ├── Images
│       └── Audio
│
├── 3. Data Understanding
│       ├── Shape
│       ├── Features
│       ├── Data types
│       └── Target
│
├── 4. Data Quality Assessment
│       ├── Missing values
│       ├── Duplicates
│       ├── Invalid values
│       ├── Outliers
│       └── Incorrect labels
│
├── 5. Data Cleaning
│       ├── Missing values
│       ├── Duplicates
│       ├── Data types
│       └── Inconsistencies
│
├── 6. Data Integration
│       ├── Merge
│       ├── Join
│       └── Concatenate
│
├── 7. Exploratory Data Analysis
│       ├── Distributions
│       ├── Relationships
│       ├── Target analysis
│       └── Visualization
│
├── 8. Feature Engineering
│       ├── New features
│       ├── Date features
│       ├── Aggregation
│       └── Domain features
│
```

```
├── 9. Feature Selection
│       ├── Relevant features
│       ├── Remove redundant features
│       └── Remove leakage
│
├── 10. Data Splitting
│       ├── Training
│       ├── Validation
│       └── Testing
│
├── 11. Preprocessing
│       ├── Encoding
│       ├── Scaling
│       ├── Normalization
│       └── Pipelines
│
├── 12. Class Balancing
│       ├── Oversampling
│       ├── Undersampling
│       ├── SMOTE
│       └── Class weights
│
├── 13. Data Augmentation
│       ├── Images
│       ├── Audio
│       └── Other modalities
│
├── 14. Model Training
│       ├── Baseline
│       ├── Candidate models
│       └── Hyperparameter tuning
│
├── 15. Model Evaluation
│       ├── Accuracy
│       ├── Precision
│       ├── Recall
│       ├── F1
│       ├── ROC-AUC
│       └── MAE/RMSE/R²
│
├── 16. Error Analysis
│       ├── Confusion matrix
│       ├── Misclassifications
│       └── Failure cases
│
├── 17. Model Interpretation
│       ├── Feature importance
│       ├── SHAP
│       └── Explainability
│
├── 18. Results Visualization
│       ├── Curves
│       ├── Confusion matrix
│       ├── Performance charts
│       └── Predictions
│
├── 19. Results Interpretation
```

```
│       ├── Findings
│       ├── Limitations
│       └── Implications
│
├── 20. Results Presentation
│       ├── Report
│       ├── Notebook
│       ├── Dashboard
│       └── Presentation
│
└── 21. Deployment / Future Work
        ├── Save model
        ├── API/application
        ├── Edge deployment
        └── Monitoring
```

The Most Important Distinction from BI

For Business Intelligence, the central flow is:

```
Raw Data → Information → Insights → Business Decision
```

For Machine Learning, the central flow is:

```
Raw Data → Clean Data → Features → Training Data → Model → Evaluation →
Prediction
```

And the most important principle throughout ML data preparation is:

> ***Good data preparation is not simply making the dataset look clean; it is making sure the model receives valid, representative, leakage-free information in a form it can learn from.***